Tarea 1

Bulk-loading de R-trees

Profesores: Benjamín Bustos y Gonzalo Navarro
Auxiliares: Claudio Gaete y Gabriel Tapia

1. Indicaciones administrativas

- Fecha de entrega: lunes 4 de mayo (atrasos hasta el domingo 10 de mayo con el descuento correspondiente).
- Grupos de a lo más 3 personas **de la misma sección**.
- Lenguaje a utilizar: C, C++ o Java.

2. Contexto

El objetivo de esta tarea es implementar dos métodos de *bulk-loading* (carga masiva de datos) para crear R-trees que se puedan consultar en memoria externa. Los R-trees guardarán puntos en dos dimensiones, y las consultas serán del tipo "retornar todos los puntos contenidos en un rectángulo R".

Para comparar el rendimiento de los métodos de *bulk-loading*, se utilizarán dos datasets, ambos disponibles en el siguiente enlace:

1. Un dataset con puntos aleatorios uniformemente distribuidos en el rango $[0, 1] \times [0, 1]$.
2. Un dataset real con las ubicaciones de edificios a lo largo de Europa (Figura 1; ver la página de descarga para la metodología de obtención de los datos). Las coordenadas de estos puntos fueron normalizadas al mismo rango $[0, 1] \times [0, 1]$.

Para ambos datasets, los puntos están guardados desordenadamente en un archivo binario, como pares de 2 `floats` (4 bytes cada uno) indicando las coordenadas X e Y de cada punto.

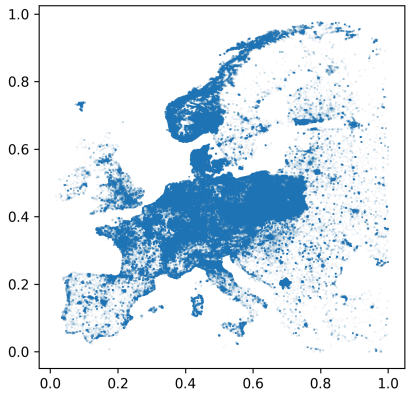

Figura 1: Dataset de Europa (normalizado)

3. R-trees

El R-tree es una estructura similar al árbol B, pero que permite guardar puntos en múltiples dimensiones (en nuestro caso, 2). Cada nodo está representado como un par clave-valor. La clave es el rectángulo más pequeño que contiene a todos los hijos del nodo (el *Minimum Bounding Rectangle*, o MBR). El valor es un puntero a la dirección donde están guardados todos los hijos; estos hijos a su vez tienen su propia clave (su MBR) y su propio valor (puntero a sus hijos). En el nivel más bajo del árbol, la clave de cada hijo es un punto (un rectángulo donde las 4 esquinas están en el mismo lugar), y el valor es un puntero nulo (pues no hay más hijos). En la página 44 del apunte se puede encontrar más información sobre los R-trees, si se desea profundizar al respecto.

Para efectos de esta tarea, cada nodo del R-tree tendrá los siguientes campos:

- `k`: un `int` (4 bytes) que guarda la cantidad de hijos actualmente contenidos en el nodo. `k` puede fluctuar entre 1 y b.
- `hijos`: un arreglo de b pares clave-valor, donde:
  - Cada clave está compuesta de 4 `floats` (16 bytes en total), representando (x_1, x_2, y_1, y_2) (los límites inferior y superior del MBR en cada coordenada). Si el nodo es una hoja, $x_1 = x_2$ e $y_1 = y_2$ (el MBR es un punto).
  - Cada valor es un `int` (4 bytes) representando la posición del hijo en el archivo (0 si es el primer nodo del archivo, 1 si es el segundo, etc.). Si el nodo es una hoja, el valor es -1 (indicando que no existe el hijo).
- `pad`: 12 bytes que quedan libres, para que el tamaño del nodo coincida con el tamaño de un bloque.

Para esta tarea, se utilizará $b = 204$ para que cada nodo tenga un tamaño exacto de 4096 bytes (el tamaño típico de un bloque en disco).

3.1. Utilización de memoria externa

Los árboles R se guardarán de manera **serializada** en un archivo binario: como se vio anteriormente, cada bloque del disco contiene exactamente un nodo, y cada nodo padre guarda las posiciones en disco de sus nodos hijos (para poder acceder a ellos cuando necesite).

Sin embargo, tener el árbol completamente en disco implica que cada inserción requiere múltiples lecturas; realizar las millones de inserciones que se requieren en la tarea toma un tiempo excesivo. Por ende, para la **creación** del R-tree, se simulará la serialización de la siguiente forma:

- El árbol será representado como un arreglo (o vector) de nodos en RAM; el primer elemento del arreglo siempre será el nodo raíz.
- Cada nodo interno guardará **el índice** en el cual está ubicado cada uno de sus hijos.

Un diagrama de esta representación se puede apreciar en la Figura 2. Una vez se haya construido el árbol, este se guardará en disco escribiendo cada uno de los nodos de forma secuencial a un archivo binario. Las consultas al árbol se deben hacer **leyendo desde el archivo binario**, que

en términos de código es muy similar a leer desde el arreglo en RAM, solo que en vez de acceder al elemento i de un arreglo, tenemos que leer desde la posición $i \cdot$ `(tamaño de un nodo en bytes)` del archivo binario.

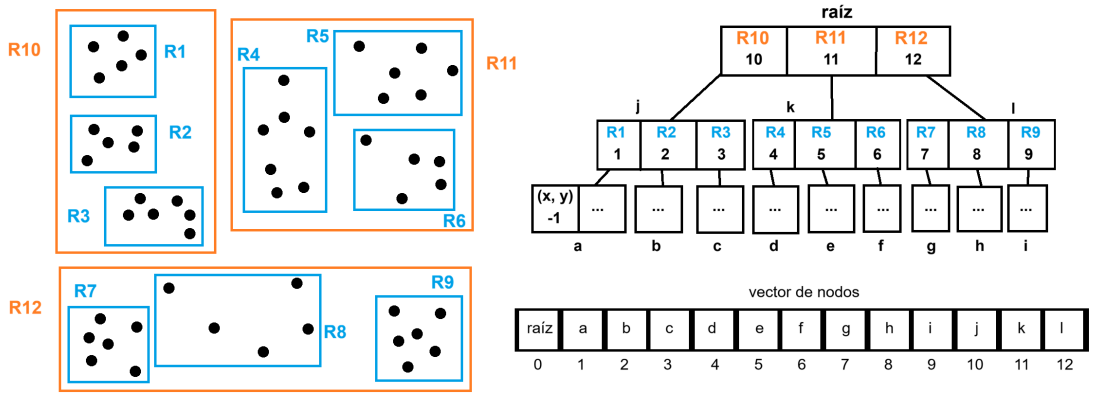

Figura 2: Representación de un R-tree. A la izquierda la distribución de los MBR y los puntos en el plano; arriba a la derecha la estructura del árbol y la información que guarda cada nodo (pares llave-valor con los MBR y los índices de cada hijo); abajo a la derecha el vector de nodos que se guarda en disco

4. Implementación

Se espera que puedan implementar mecanismos para construir un R-tree mediante *bulk--loading*, y para realizar consultas de todos los puntos que caen dentro de un rectángulo $R = \{x_{\min}, x_{\max}, y_{\min}, y_{\max}\}$. **No se debe implementar la inserción ni borrado de elementos individuales**. Recuerden que para el *bulk-loading*, se guarda el árbol en un arreglo de nodos en RAM, mientras que para la búsqueda, los nodos se deben leer desde un archivo en disco.

4.1. *Bulk-loading*

Se solicita la implementación de los siguientes métodos de *bulk-loading*:

4.1.1. Nearest-X

Dado un conjunto de n pares llave-valor, donde las llaves son rectángulos (MBRs) y los valores son los índices correspondientes al nodo hijo, hacemos lo siguiente:

1. Ordenar los rectángulos según la coordenada X del centro del rectángulo.
2. Juntar en $\frac{n}{b}$ grupos de rectángulos consecutivos; cada uno de estos grupos formará un nodo con b elementos (el último nodo puede tener menos).
3. Guardar los $\frac{n}{b}$ nodos creados en el vector de nodos, **dejando el primer espacio vacío** (la raíz va ahí).

4. Para cada uno de los $\frac{n}{b}$ nodos creados, calcular su MBR y generar $\frac{n}{b}$ pares llave-valor, donde nuevamente las llaves son rectángulos y los valores son los índices asociados a cada nodo (la posición en la que se guardaron en el vector de nodos).
5. Si los $\frac{n}{b}$ pares llave-valor entran en un nodo (es decir, $\frac{n}{b} \leq b$), entonces creamos el nodo raíz con todos los pares llave-valor y lo guardamos en la primera posición del vector de nodos.
   - Si no, entonces aplicamos Nearest-X recursivamente a los $\frac{n}{b}$ pares, generando un nivel más en el R-tree.

El algoritmo inicia con $n = N$ pares llave-valor, donde cada llave es un punto (rectángulo de área 0, donde $x_1 = x_2, y_1 = y_2$) y cada índice es -1 (es decir, no se tiene un nodo hijo en el vector de nodos).

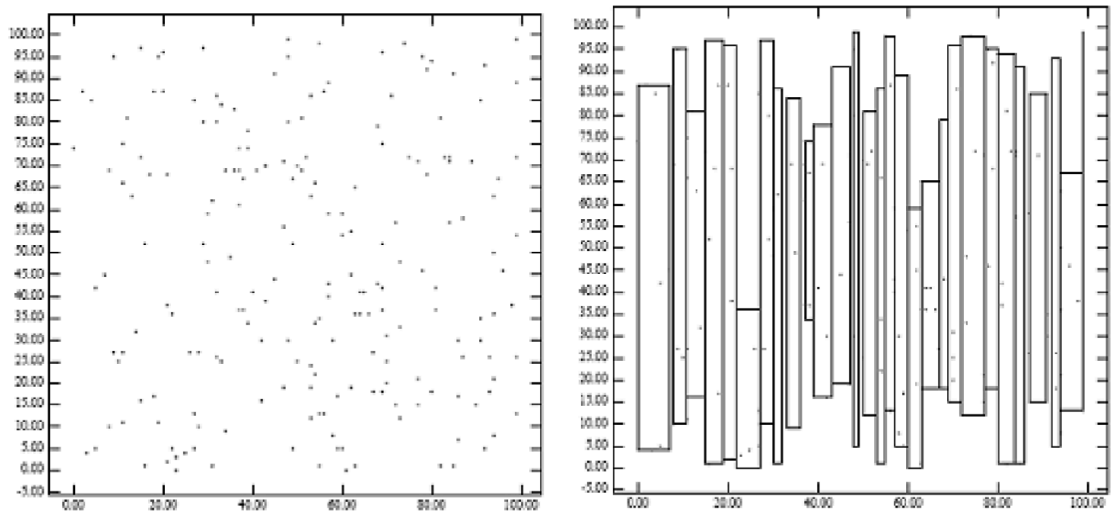

Figura 3: Representación de la primera iteración de Nearest-X. A la izquierda, los puntos originales; a la derecha, los nodos creados agrupando los puntos según su coordenada X

4.1.2. Sort-Tile-Recursive (STR)

Dado un conjunto de n pares llave-valor, donde las llaves son rectángulos (MBRs) y los valores son los índices correspondientes al nodo hijo, hacemos lo siguiente:

1. Ordenar los rectángulos según la coordenada X del centro del rectángulo.
2. Juntar en $S = \sqrt{\frac{n}{b}}$ grupos de rectángulos consecutivos.
3. Para cada grupo:
   1. Ordenar los rectángulos según la coordenada Y del centro del rectángulo.
   2. Dividir nuevamente en $S = \sqrt{\frac{n}{b}}$ grupos. En total, vamos a tener $S^2 = \frac{n}{b}$ grupos; cada uno de estos grupos formará un nodo con b elementos.
4. Repetir los pasos 3 a 5 de Nearest-X (haciendo la recursión con STR si no entran todos los pares llave-valor en un único nodo raíz).

Al igual que Nearest-X, el algoritmo inicia con $n = N$ pares llave-valor, donde cada llave es un punto (rectángulo de área 0, donde $x_1 = x_2, y_1 = y_2$) y cada índice es -1 (es decir, no se tiene un nodo hijo en el vector de nodos).

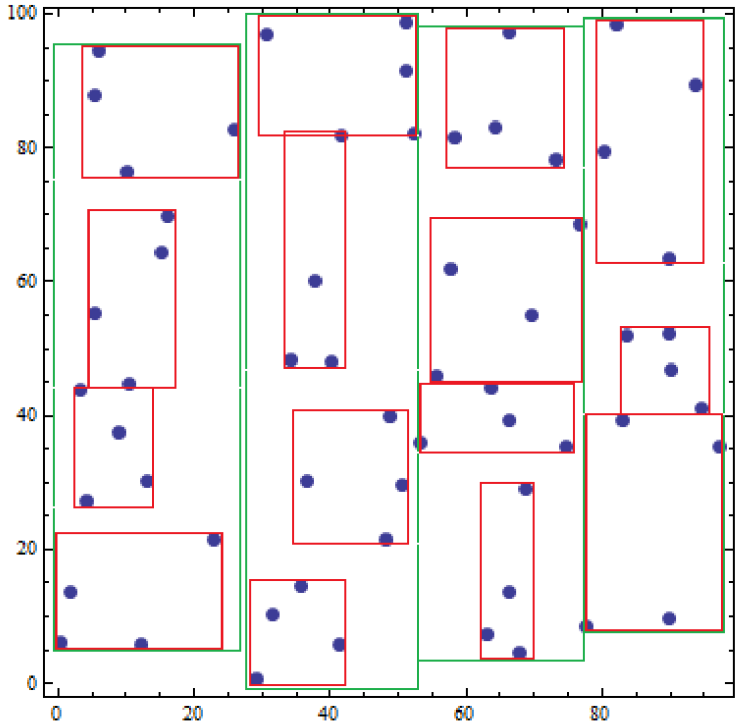

Figura 4: Representación de la primera iteración de STR; con $S = 4$. Los rectángulos verdes se obtienen ordenando según la coordenada X, pero **no** son nodos; los rectángulos rojos se obtienen ordenando los puntos de cada rectángulo verde según la coordenada Y, y estos **sí** equivalen a nodos en el R-tree

4.2. Consultas

Para realizar una consulta del tipo "retornar los puntos contenidos en un rectángulo R", se inicia leyendo desde el nodo raíz (posición 0 en el archivo binario). Para cada hijo, si su MBR intersecta con R, se lee el nodo hijo y se realiza la consulta recursivamente. Al llegar a una hoja (caso base), se retornan todos los puntos del nodo que estén dentro de R.

5. Experimentación

5.1. Construcción

Para cada $N \in \{2^{15}, 2^{16}, ..., 2^{24}\}$, realizar lo siguiente:

- Crear 4 R-trees mediante *bulk-loading*:
  - Dataset aleatorio (los primeros N elementos), usando Nearest-X.
  - Dataset aleatorio (los primeros N elementos), usando STR.

- Dataset de Europa (los primeros N elementos), usando Nearest-X.
- Dataset de Europa (los primeros N elementos), usando STR.

Deberán reportar y graficar el tiempo de creación de cada árbol en función de N.

5.2. Consultas

Para las consultas, usaremos los 4 R-trees con $N = 2^{24}$ elementos que se construyeron en la parte anterior. Estos árboles deben ser escritos a disco (escribiendo cada nodo de manera secuencial). Consultas realizadas a árboles guardados en RAM **no serán consideradas correctas**.

Para cada $s = \{0.0025, 0.005, 0.01, 0.025, 0.05\}$:

- Generar 100 cuadrados aleatorios, con lados de largo s y en el rango $[0, 1] \times [0, 1]$.
- Para cada árbol, consultar por los puntos contenidos en cada uno de los 100 cuadrados generados.

Para cada árbol, deberán reportar y graficar:

- La cantidad promedio de lecturas a disco en cada consulta (cómo varía en función de s).
- La cantidad promedio de puntos encontrados (cómo varía en función de s), con barras de error que muestren la desviación estándar (la desviación será baja si las 100 consultas retornaron un número similar de puntos, y será alta si algunas consultas retornaron muchos puntos y otras muy pocos).

5.3. Bonificación (+1.5 pts)

Se puede optar a una bonificación de 1.5 pts, sumados a la **nota de código**; si esta ya es un 7, se pueden sumar a la **nota de informe**. Esta bonificación **no es transferible a otras evaluaciones**.

Se debe construir un R-tree usando el archivo `europa_bonus.bin` disponible en el siguiente enlace este dataset contiene los mismos datos de Europa, pero en su versión no normalizada, por lo cual las coordenadas X e Y representan longitud y latitud, respectivamente. El rango de este dataset es $[-11, 35] \times [35, 72]$.

Para optar a la bonificación, deben realizar una consulta alrededor de una ubicación que ustedes decidan, y visualizar los puntos obtenidos (utilizando un *scatterplot*). Adjunten en su informe una imagen de los puntos obtenidos, junto con una descripción de la ubicación (p. ej. "Edificios en los alrededores de Barcelona, España").

6. Entregables

Se deberá entregar el código y un informe donde se explique el experimento en estudio. Con esto se obtendrá una nota de código ($NCod$) y una nota de informe ($NInf$). La nota final de la tarea será el promedio simple entre ambas notas ($NT_1 = 0.5 \cdot NCod + 0.5 \cdot NInf$).

6.1. Código

La entrega de código tiene que contener:

- **(0.3 pts)** README: Archivo con las instrucciones para ejecutar el código, debe ser lo suficientemente explicativo para que cualquier persona solo leyendo el README pueda ejecutar la totalidad de su código (incluyendo librerías no entregadas por el equipo docente que potencialmente se deban instalar).
- **(0.2 pts)** Firmas: Cada estructura de datos y función debe tener una descripción de lo que hace y una descripción de sus parámetros de entrada y salida.
- **(1.0 pts)** Uso de disco: El uso de disco debe ser correcto.
  - Al crear el árbol R-tree, se deben guardar los nodos en un arreglo secuencial, simulando el disco.
  - Una vez completo el árbol, se debe guardar en un archivo binario antes de realizar consultas.
  - Se debe poder realizar la interpretación del archivo binario en memoria principal para realizar las búsquedas.
  - Durante las consultas, se deben realizar lecturas por bloque (recuerden que un bloque es equivalente a un nodo para efectos de esta tarea).
- **(1.5 pts)** Implementación de *bulk-loading*: La implementación de Nearest-X y STR son correctas.
- **(1.0 pts)** Implementación de búsqueda: La implementación de consultas por rectángulo es correcta.
- **(1.0 pts)** Experimento: Se realiza la experimentación para los valores de N y s pedidos.
- **(0.5 pts)** Obtención de resultados: La forma en el que se obtienen los resultados (tiempos de ejecución, I/Os y cantidad de puntos retornados por consulta) es correcta.
- **(0.5 pts)** Main: Un archivo o parte del código (función main) que permita ejecutar la construcción y búsquedas.
- Bonus **(1.5 pts)** Visualización de consulta: se realiza lo pedido en la Sección 5.3.

6.2. Informe

El informe debe ser claro y conciso. Se recomienda hacerlo en LaTeX o Typst. Debe contener:

- **(0.8 pts)** Introducción: Presentar el tema en estudio, resumir lo que se dirá en el informe y presentar una hipótesis.
- **(0.8 pts)** Desarrollo: Presentación de algoritmos, estructuras de datos, cómo funcionan y por qué. Recordar que los métodos ya son conocidos por el equipo docente, lo que importa son sus propias implementaciones (qué decisiones tomaron que no están mencionadas en el enunciado).
- **(2.4 pts)** Resultados: Especificación de los datos que se utilizaron para los experimentos, la cantidad de veces que se realizaron los tests, con qué inputs, qué tamaño, etc. Se debe mencionar el sistema operativo y los tamaños de sus cachés y RAM con los que se ejecutaron los experimentos. Se deben mostrar gráficos/tablas con la información solicitada en la Sección 5 y mencionar solo lo que se puede observar de estos.
- **(1.2 pts)** Análisis: Comentar y concluir sus resultados. Se hacen las inferencias de sus resultados, aplicando conocimientos del curso.

- **(0.8 pts)** Conclusión: Recapitulación de lo que se hizo, se concluye lo que se puede decir con respecto a sus resultados. También ven si su hipótesis se cumplió o no y analizan la razón. Por último, se menciona qué se podría mejorar en su desarrollo en una versión futura, qué falta en su documento, qué no se ha resuelto y cómo se podrían extender.

7. Informaciones útiles

- Si hacen búsquedas usando cuadrados de largo 0.01 en el dataset aleatorio, debieran obtener un 0.01% de los datos en cada búsqueda. Si están obteniendo muchos menos datos que eso, significa que su búsqueda no está funcionando adecuadamente (o construyeron mal el árbol).
- Para contabilizar las lecturas durante las consultas, les recomendamos crear una función `readNode`, que lea un nodo del disco (con un cierto *offset*) y automáticamente sume 1 a un contador de lecturas.
- Si están usando C++, les puede ser útil el siguiente repositorio creado por Pablo Skewes (auxiliar durante Otoño 2025). El proyecto consiste en serializar un árbol de búsqueda binaria, por lo cual comparte similitudes con lo que se les pide en esta tarea (p. ej. al leer y escribir estructuras en disco). También les puede servir como referencia en cuanto a calidad del README y las firmas.